

沈阳航空航天大学

学生实验报告



课程名称	数据分析编程基础 B
实验名称	实验四 科学计算可视化实验
专业班级	物联网 2202
学生学号	223428010210
学生姓名	陈梓欣
指导教师	李济瀚
实验时间	2023-2024 学年第二学期
实验地点	机械馆 410-1

一、实验目的

熟悉实验环境，分析和了解 numpy 库和 matplotlib 库的程序可视化编写与运用；培养学生函数在程序可视化编写过程中逻辑分析与实际动手的编程能力。

二、实验要求

1. 实验学时：2 学时。
2. 使用 Python 环境编写和调试程序。
3. 了解 numpy 库的可视化编写与使用。
4. 了解 matplotlib 库的可视化编写与使用。
5. 实验应通过调试，并获得正确的结果。
6. 对程序中重要的内容做必要的注释。

三、实验准备

文件的打开与读取的方法。

四、实验程序（要求：程序运行结果，程序代码写注释）

一、编程题

1. 读取 iris 数据集中鸢尾花的萼片，花瓣长度信息。

要求：对其进行求和，累计和，均值，标准差、方差、最大值和最小值。（数据集中 sepal:萼片、petal:花瓣、species:种类），（数据集中 species:种类、setosa:山鸢尾、versicolor:变色鸢尾、virginica: 维吉尼亚鸢尾）。

程序代码及注释：

```
import pandas as pd
import numpy as np

# 读取 Iris 数据集
iris = pd.read_csv('experiment004/iris.csv')

# 选择萼片(sepal)和花瓣(petal)的长度
sepal_length = iris['sepal_length']
petal_length = iris['petal_length']

# 计算所需的统计信息
def calculate_statistics(data):
    stats = {
        'sum': np.sum(data),
        'cumulative_sum': np.cumsum(data).tolist(), # 累计和返回列表
```



```

        'mean': np.mean(data),
        'std': np.std(data),
        'variance': np.var(data),
        'max': np.max(data),
        'min': np.min(data)
    }
    return stats

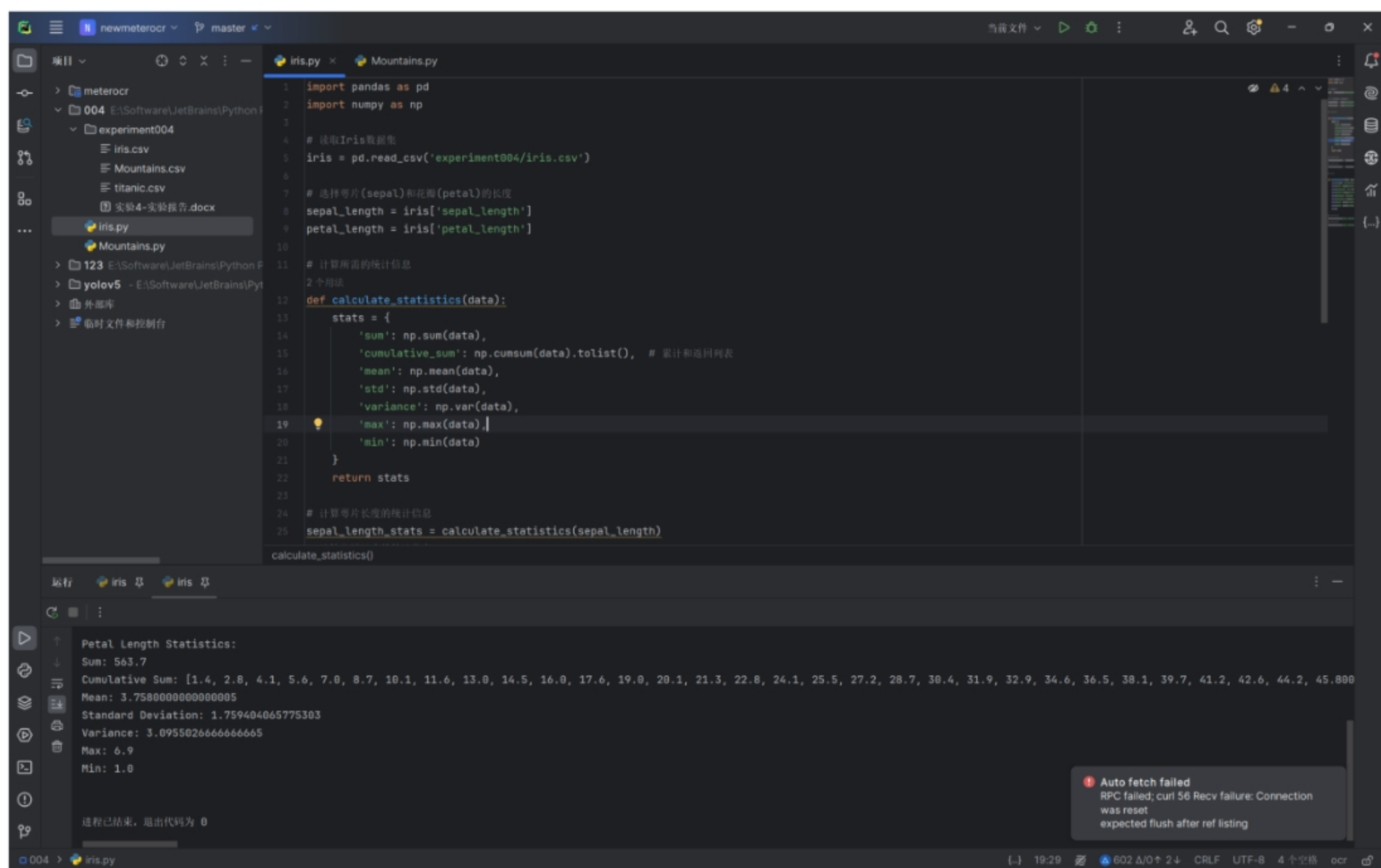
# 计算萼片长度的统计信息
sepal_length_stats = calculate_statistics(sepal_length)
# 计算花瓣长度的统计信息
petal_length_stats = calculate_statistics(petal_length)

# 输出统计信息
def print_statistics(name, stats):
    print(f"{name} Statistics:")
    print(f"Sum: {stats['sum']}")
    print(f"Cumulative Sum: {stats['cumulative_sum']}")
    print(f"Mean: {stats['mean']}")
    print(f"Standard Deviation: {stats['std']}")
    print(f"Variance: {stats['variance']}")
    print(f"Max: {stats['max']}")
    print(f"Min: {stats['min']}")
    print()

# 输出萼片长度的统计信息
print_statistics("Sepal Length", sepal_length_stats)
# 输出花瓣长度的统计信息
print_statistics("Petal Length", petal_length_stats)

```


程序运行结果：



The screenshot shows a code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code in the center is a Python script named `iris.py` that reads the Iris dataset from a CSV file and calculates various statistics for the petal length. The terminal at the bottom shows the output of the script, which includes the petal length statistics: Sum, Cumulative Sum, Mean, Standard Deviation, Variance, Max, and Min. The output is as follows:

```
Petal Length Statistics:  
Sum: 563.7  
Cumulative Sum: [1.4, 2.8, 4.1, 5.6, 7.0, 8.7, 10.1, 11.6, 13.0, 14.5, 16.0, 17.6, 19.0, 20.1, 21.3, 22.8, 24.1, 25.5, 27.2, 28.7, 30.4, 31.9, 32.9, 34.6, 36.5, 38.1, 39.7, 41.2, 42.6, 44.2, 45.898]  
Mean: 3.7580000000000005  
Standard Deviation: 1.759404065775303  
Variance: 3.0955026666666665  
Max: 6.9  
Min: 1.0
```

The terminal also shows the message "Auto fetch failed" and "RPC failed; curl 56 Recv failure: Connection was reset expected flush after ref listing".

图 1 打印信息

2. 读取 titanic 数据集中的乘客的性别、年龄等信息。

要求：实现显示其前 50 行的数据信息，实现调用函数显示数据信息总览，并实现数据中生还者的饼图绘制，利用均值填充数据中“年龄”的缺失值实现。

程序代码及注释：

```
import pandas as pd

import matplotlib.pyplot as plt

# 读取 Titanic 数据集

titanic = pd.read_csv('experiment004/titanic.csv')

# 调整 pandas 显示设置，确保显示所有列

pd.set_option('display.max_columns', None) # 显示所有列

pd.set_option('display.width', 1000) # 设置显示宽度

pd.set_option('display.max_colwidth', None) # 设置每列的最大宽度

# 显示前 50 行数据

print("前 50 行的数据信息：")

print(titanic.head(50))

# 定义一个函数显示数据信息总览

def data_overview(df):

    print("\n 数据信息总览：")

    print(df.info())

    print("\n 数据的描述性统计：")

    print(df.describe(include='all'))

# 调用函数显示数据信息总览

data_overview(titanic)

# 用均值填充年龄的缺失值
```



```
titanic['age'].fillna(titanic['age'].mean(), inplace=True)
```

```
# 绘制生还者的饼图
```

```
survived_counts = titanic['survived'].value_counts()
```

```
plt.figure(figsize=(8, 6))
```

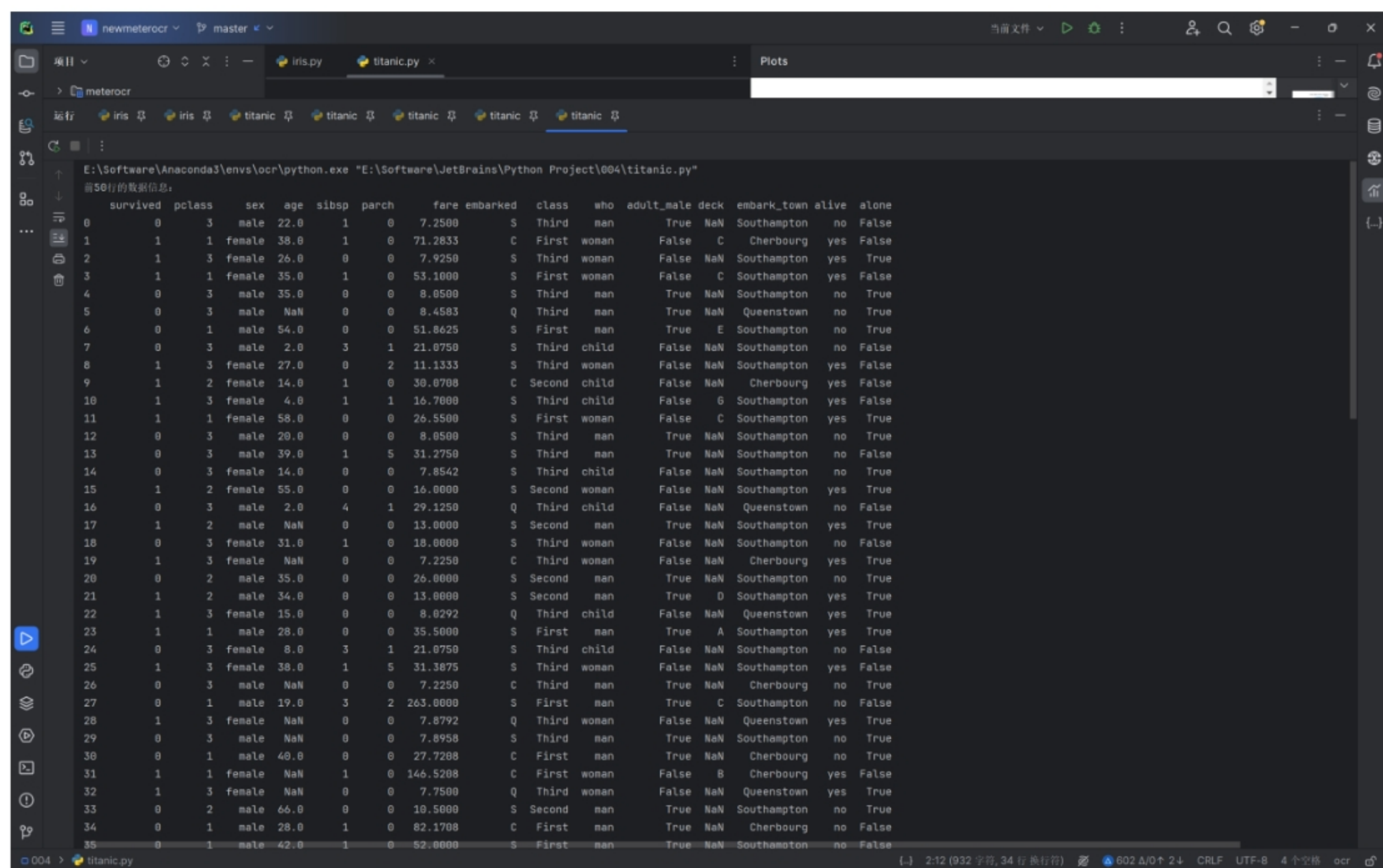
```
plt.pie(survived_counts, labels=['Not Survived', 'Survived'], autopct='%1.1f%%', startangle=140,  
colors=['lightcoral', 'lightskyblue'])
```

```
plt.title('Survival Rate of Titanic Passengers')
```

```
plt.axis('equal') # 确保饼图为圆形
```

```
plt.show()
```

程序运行结果：



	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True
5	0	3	male	NaN	0	0	8.4500	Q	Third	man	True	NaN	Queenstown	no	True
6	0	1	male	54.0	0	0	51.8625	S	First	man	True	E	Southampton	no	True
7	0	3	male	2.0	3	1	21.0750	S	Third	child	False	NaN	Southampton	no	False
8	1	3	female	27.0	0	2	11.1333	S	Third	woman	False	NaN	Southampton	yes	False
9	1	2	female	14.0	1	0	30.0700	C	Second	child	False	NaN	Cherbourg	yes	False
10	1	3	female	4.0	1	1	16.7000	S	Third	child	False	G	Southampton	yes	False
11	1	1	female	58.0	0	0	26.5500	S	First	woman	False	C	Southampton	yes	True
12	0	3	male	20.0	0	0	8.9500	S	Third	man	True	NaN	Southampton	no	True
13	0	3	male	39.0	1	5	31.2750	S	Third	man	True	NaN	Southampton	no	False
14	0	3	female	14.0	0	0	7.8542	S	Third	child	False	NaN	Southampton	no	True
15	1	2	female	55.0	0	0	16.0000	S	Second	woman	False	NaN	Southampton	yes	True
16	0	3	male	2.0	4	1	29.1250	Q	Third	child	False	NaN	Queenstown	no	False
17	1	2	male	NaN	0	0	13.0000	S	Second	man	True	NaN	Southampton	yes	True
18	0	3	female	31.0	1	0	18.0000	S	Third	woman	False	NaN	Southampton	no	False
19	1	3	female	NaN	0	0	7.2250	C	Third	woman	False	NaN	Cherbourg	yes	True
20	0	2	male	35.0	0	0	26.0000	S	Second	man	True	NaN	Southampton	no	True
21	1	2	male	34.0	0	0	13.0000	S	Second	man	True	D	Southampton	yes	True
22	1	3	female	15.0	0	0	8.0292	Q	Third	child	False	NaN	Queenstown	yes	True
23	1	1	male	28.0	0	0	35.5000	S	First	man	True	A	Southampton	yes	True
24	0	3	female	8.0	3	1	21.0750	S	Third	child	False	NaN	Southampton	no	False
25	1	3	female	38.0	1	5	31.3875	S	Third	woman	False	NaN	Southampton	yes	False
26	0	3	male	NaN	0	0	7.2250	C	Third	man	True	NaN	Cherbourg	no	True
27	0	1	male	19.0	3	2	263.0000	S	First	man	True	C	Southampton	no	False
28	1	3	female	NaN	0	0	7.8792	Q	Third	woman	False	NaN	Queenstown	yes	True
29	0	3	male	NaN	0	0	7.8958	S	Third	man	True	NaN	Southampton	no	True
30	0	1	male	40.0	0	0	27.7208	C	First	man	True	NaN	Cherbourg	no	True
31	1	1	female	NaN	1	0	146.5208	C	First	woman	False	B	Cherbourg	yes	False
32	1	3	female	NaN	0	0	7.7500	Q	Third	woman	False	NaN	Queenstown	yes	True
33	0	2	male	66.0	0	0	10.5000	S	Second	man	True	NaN	Southampton	no	True
34	0	1	male	28.0	1	0	82.1708	C	First	man	True	NaN	Cherbourg	no	False
35	0	1	male	42.0	1	0	52.0000	S	First	man	True	NaN	Southampton	no	False

图 2 打印信息(1)

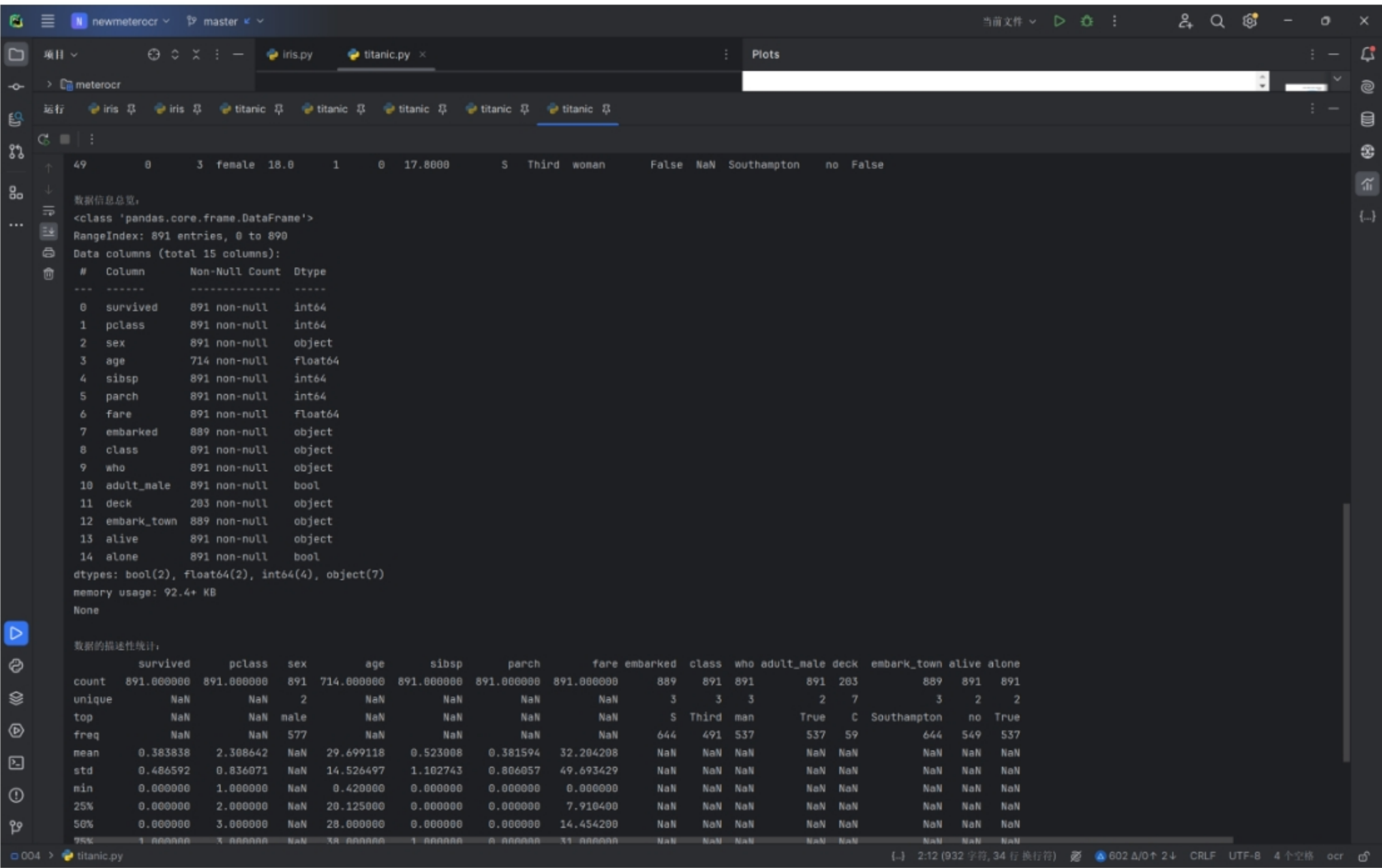


图 3 打印信息(2)

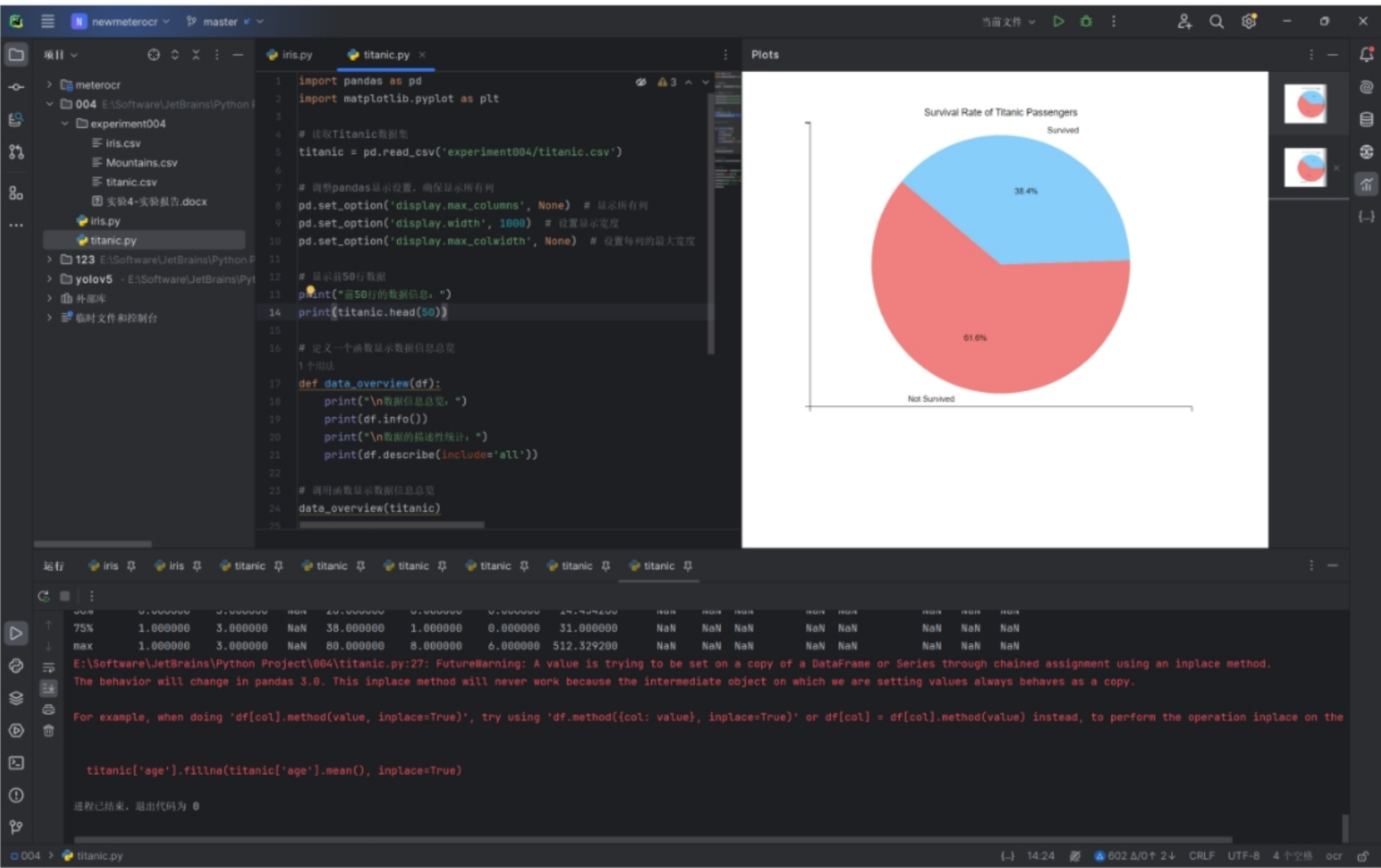


图 4 输出饼状图

3. 读取 Mountains 数据集的信息，实现山峰可视化功能。

要求：编写过程中要求设置图片显示的主题样式，显示中文指定“SimHei”字体，可以运用 def 函数形式编写，实现对数据集中“Height(m)”：“Height”，“Ascents bef.2004”：“Success”，“Failed attempts bef.2004”：“Failed”重构、清洗（用 0 填充“成功”或者“失败”的缺失项）以及可视化功能，用图形的方式显示山峰的数量和海拔，显示山峰的数量和年份，用子图形式显示海拔与登顶、登顶与排名。

程序代码及注释：

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

import re

# 读取 Mountains 数据集，尝试使用不同的编码

try:

    mountains = pd.read_csv('experiment004/mountains.csv', encoding='utf-8')

except UnicodeDecodeError:

    mountains = pd.read_csv('experiment004/mountains.csv', encoding='latin1')

# 打印列名以确认其正确性

print("列名：", mountains.columns)

# 调整 pandas 显示设置，确保显示所有列

pd.set_option('display.max_columns', None)

pd.set_option('display.width', 1000)

pd.set_option('display.max_colwidth', None)

# 设置图片显示的主题样式

sns.set_theme(style="whitegrid")
```



```

# 设置中文字体为 SimHei

plt.rcParams['font.sans-serif'] = ['SimHei']

plt.rcParams['axes.unicode_minus'] = False


# 清洗数据集中的非数字值

def clean_numeric_values(series):

    # 移除所有非数字字符

    series = series.apply(lambda x: re.sub(r'\D', "", str(x)))

    # 将清理后的值转换为整数

    series = series.apply(lambda x: int(x) if x.isdigit() else 0)

    return series


# 重构和清洗数据集

def clean_and_restructure_data(df):

    df = df.rename(columns={

        'Height (m)': 'Height',

        'Ascents bef. 2004': 'Success',

        'Failed attempts bef. 2004': 'Failed'

    })

    df['Success'] = clean_numeric_values(df['Success'])

    df['Failed'] = clean_numeric_values(df['Failed'])

    return df


# 检查重命名是否成功

mountains = clean_and_restructure_data(mountains)

print("重命名后的列名: ", mountains.columns)

```


可视化功能

```
def visualize_data(df):
```

```
    plt.figure(figsize=(14, 10))
```

```
    # 图 1: 山峰的数量和海拔
```

```
    plt.subplot(2, 2, 1)
```

```
    sns.histplot(df['Height'], bins=30, kde=True)
```

```
    plt.title('山峰的数量和海拔')
```

```
    plt.xlabel('海拔 (m)')
```

```
    plt.ylabel('数量')
```

```
    # 图 2: 山峰的数量和年份 (假设有 Year 列)
```

```
    if 'Year' in df.columns:
```

```
        plt.subplot(2, 2, 2)
```

```
        sns.histplot(df['Year'], bins=30, kde=True)
```

```
        plt.title('山峰的数量和年份')
```

```
        plt.xlabel('年份')
```

```
        plt.ylabel('数量')
```

```
    # 图 3: 海拔与登顶
```

```
    plt.subplot(2, 2, 3)
```

```
    sns.scatterplot(x='Height', y='Success', data=df)
```

```
    plt.title('海拔与登顶')
```

```
    plt.xlabel('海拔 (m)')
```

```
    plt.ylabel('成功次数')
```

```
    # 图 4: 登顶与排名 (假设有 Rank 列)
```

```
    if 'Rank' in df.columns:
```

```
        plt.subplot(2, 2, 4)
```



```
sns.scatterplot(x='Success', y='Rank', data=df)
```

```
plt.title('登顶与排名')
```

```
plt.xlabel('成功次数')
```

```
plt.ylabel('排名')
```

```
plt.tight_layout(pad=4.0, w_pad=5.0, h_pad=5.0)
```

```
plt.show()
```

```
visualize_data(mountains)
```

程序运行结果：

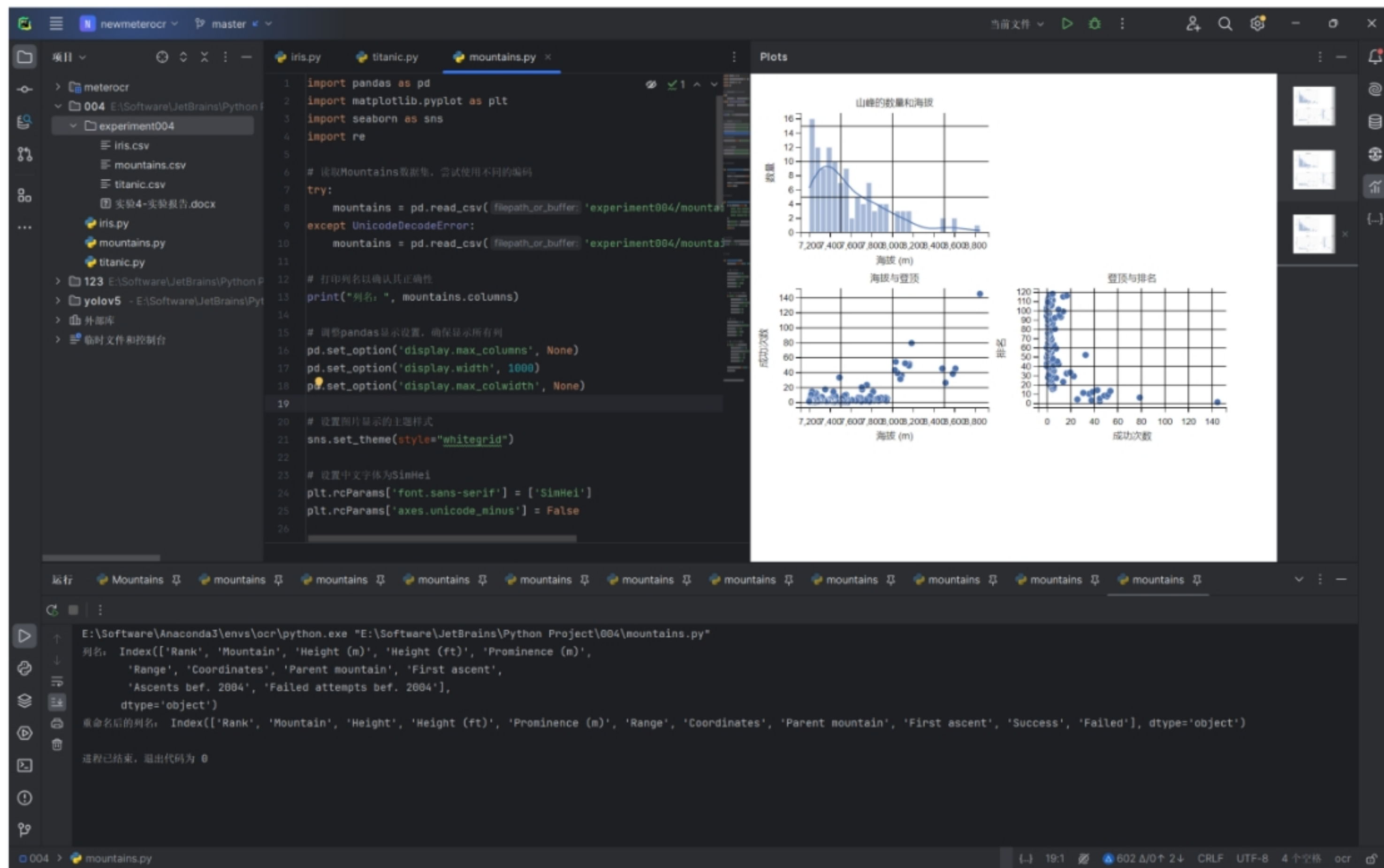


图 5 打印信息

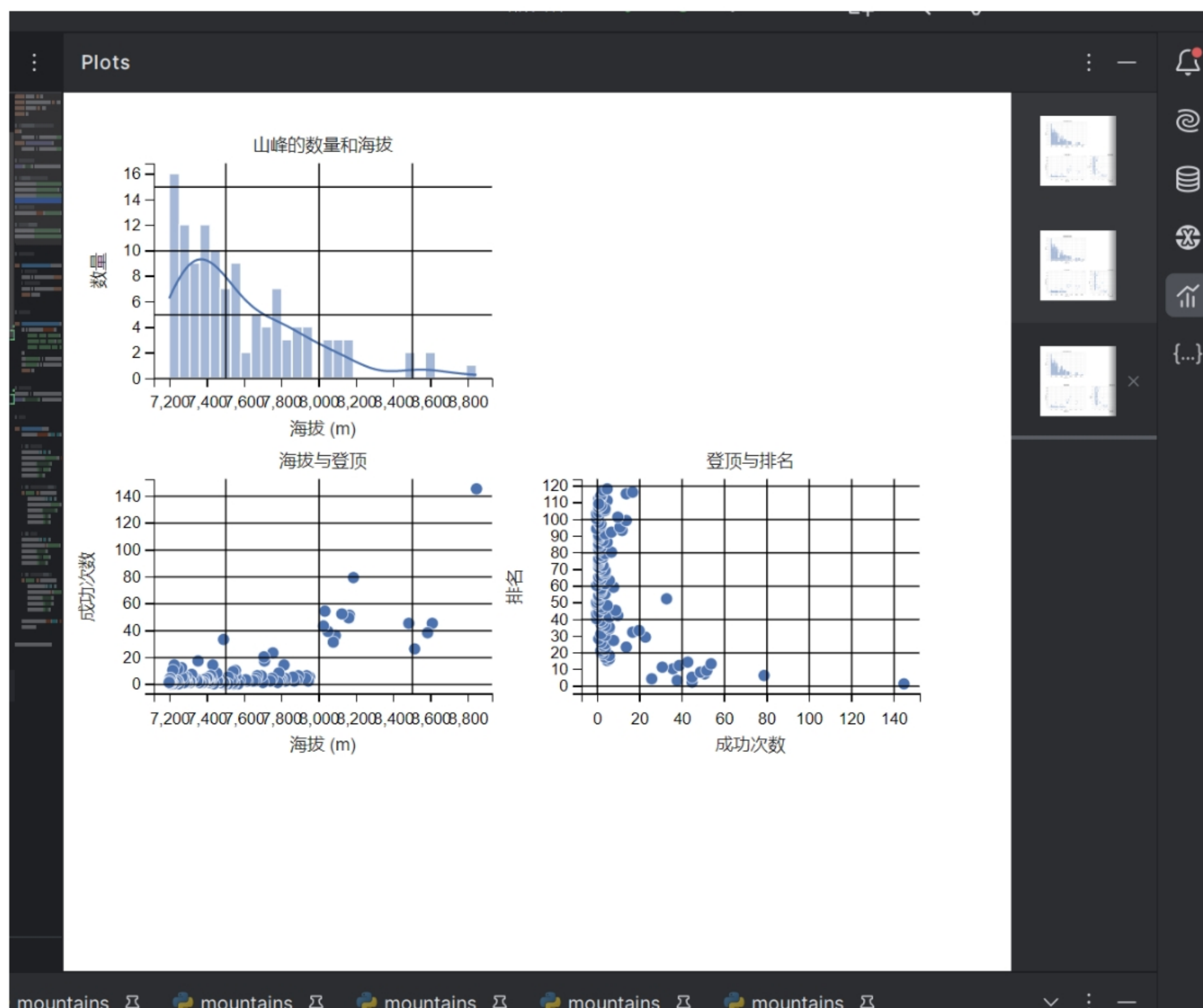


图 6 绘制子图

